Stack-Disciplined Open Addressing: Exact Undo, Exact Refusal Probabilities, and Key-Lifetime Rules for Bounded-Memory Scoped Tables

Shubhi Singh Adhyan Jain
Vellore Institute of Technology, Vellore, India

Abstract

Scoped symbol tables release declarations in strict last-in-first-out order. We ask what this discipline gives an open-addressing hash table that must live in one fixed buffer, must refuse a request cleanly when it is full, and must bound the work of every call. *(1)* For every non-reordering open-addressing scheme, deletion in stack order by clearing one cell is an exact undo: the table is at all times identical to the insertion-only table of the live sequence, so insertion-only analyses of the table's state, and of the cost of the next operation, apply unchanged. For greedy schemes the deletion orders for which clearing is safe, for a given table, are exactly the linear extensions of a dependency relation between keys, and stack order is the *only* order that is safe for every hash assignment; in linear probing, clearing one uniformly random key of an insertion-built table orphans on average $\alpha/(2(1-\alpha))$ live keys (in the limit), with an explicit law (a discrete M/D/1 queue in the limit) under which only a fraction $(1-\alpha)(e^{\alpha}-1)/\alpha$ of the keys can be removed out of order without harm. *(2)* For a table that refuses an insertion needing more than L probes, the refusal probability is the tail of the classical unsuccessful-search cost of linear probing; we give an elementary derivation of its explicit finite-size form and use it as a design tool. The Chernoff-type bound used previously overstates it by a factor that grows like $L^{3/2}$; at load 3/4 the limit needed for a per-insertion refusal probability of 10^{-6} is 258 rather than 455 for $m = 2^{16}$ (260 in the limit $m \to \infty$), and far smaller for small tables. *(3)* The guarantee of a keyed hash holds against adversarial names only if the key's lifetime does not exceed the table's: with a persistent key and an oracle that reveals home slots an adversary builds arbitrarily long runs with $O(m \log m)$ queries, and with only a probe-length oracle it learns one colliding name per m queries. *(4)* Exact space-ledger identities show that on eight compiler traces the 16–18% saving of the compact layout over a conventional one is a stored hash word (9.2–12.2%) plus name packing (5.3–9.0%); interning itself is space-neutral. Combined with the insertion-only bounds of Farach-Colton, Krapivin and Kuszmaul, the exact transfer yields the optimal probe complexity of open addressing without reordering under stack-disciplined workloads, $\Theta(\log \delta^{-1})$ non-greedy and $\Theta(\log^2 \delta^{-1})$ greedy at load $1-\delta$, both worst-case and amortised: strictly harder than insertion-only workloads (amortised $O(1)$) and exponentially easier than random deletions ($\delta^{-\Omega(1)}$). Results are checked by exhaustive enumeration, exact rational arithmetic, and a statistical test against a real implementation.

Keywords: open addressing; linear probing; stack discipline; symbol tables; bounded memory; hash flooding; exact analysis.

1 Introduction

A scoped symbol table binds identifiers to declarations and forgets the declarations of a block when the block ends. The textbook implementation is a hash table from names to their innermost declaration plus an undo stack that restores what a scope hid [3, 16]. When the compiler or interpreter that owns the table runs on a microcontroller, in a boot loader, or under a fixed allocation, three further requirements appear: the table must live in *one fixed buffer*,

it must *refuse* a request that does not fit and stay usable, and the *work* of one call must be bounded even when the source text is supplied by someone else. STRATA [35] is a design that meets them: one buffer holds an open-addressing index, one packed record per visible name, and the undo stack, and an operation that does not fit changes nothing.

An earlier, empirical study of STRATA reported what that design costs and saves on events logged by real compilers. This article asks a different question: *what does the stack discipline itself imply?* Scope exit removes declarations in exactly the reverse order of their creation. That is a very special deletion order for an open-addressing table, and, as we show, it makes four things exact that are otherwise hard or open: the state of the table after deletions (Section 3), the probability that a probe-limited table refuses an honest name (Section 4), what an adaptive adversary can do (Section 5), and where the memory goes (Section 6).

Contributions.

C1. Exact undo and its converse (Section 3). For every non-reordering open-addressing scheme, popping the newest key by clearing its cell restores the previous table exactly (Theorem 2); hence, in the random-hash model, the table at every moment has the law of an insertion-only table, and every insertion-only result about the table's state or the cost of the next operation transfers (Corollary 3). For a given table, the deletion orders for which clearing is safe are exactly the linear extensions of the dependency relation "y's probe path passes through the cell of x" (Theorem 4(a)); stack order is the *only* order that is safe for all hash assignments (Theorem 4(b)). The number of keys that depend on a key is the length of a queue minus one: it has an explicit finite-size law and, as $n/m \to \alpha$, the law of a discrete M/D/1 queue, so that clearing a random live key orphans on average the mean displacement of a key and only a fraction $(1-\alpha)(e^{\alpha}-1)/\alpha$ of the keys can be removed out of order without harm (Proposition 5). Combined with the bounds of Farach-Colton, Krapivin and Kuszmaul on open addressing without reordering, the exact transfer gives tight probe complexity for stack-disciplined workloads: $\Theta(\log \delta^{-1})$ and $\Theta(\log^2 \delta^{-1})$ (non-greedy and greedy), both worst-case and amortised, strictly between the $O(1)$ amortised optimum for insertions only and the $\delta^{-\Omega(1)}$ optimum for random deletions (Theorem 6).

C2. Exact refusal probabilities for probe-limited tables (Section 4). For linear probing on a ring with n keys among m cells, the probability that an independent new key needs more than L probes is the tail of the unsuccessful-search cost, whose exact generating function and Poisson limit are known [22, 24]. We give an elementary, self-contained derivation of an explicit finite-size sum (Theorem 7) and its tail $\sim C\rho^L L^{-3/2}$ (Corollary 8); what is new is the use: the Chernoff-type bound used for probe limits loses a factor $\Theta(L^{3/2})$, the exact values give smaller, tabulated limits (Table 4), and the formula is validated against a real probe-limited implementation. For a monotone fill we also give the exact probability that no push is refused, with a closed-form generating function (Theorem 10); it shows that the union bound over the whole run overstates the needed limit by about a quarter.

C3. Key-lifetime rules (Section 5). With an independent key per table, the exact law holds against arbitrary adversarial names (Proposition 11); with a persistent key and a probe oracle it does not (Proposition 12), and we quantify the cost of the attack.

C4. Exact space ledger (Section 6). Identities for the words used by five single-buffer layouts at every instant (Proposition 14), evaluated on eight compiler traces; they decompose the saving of the compact layout and show that interning contributes none of it on these programs.

What is not claimed. The undo observation is elementary; its value lies in what follows from it, in the characterisation of the safe orders, and in the orphan law. The queue representation of linear probing and the limit law of the orphan count (the Robin Hood displacement law) are due to Janson [21]; the single-deletion form of the safe-order condition is Sanders' [34]. We did not find the reading of that law as the number of keys orphaned by a deletion, the exact finite-size form, the characterisation of safe deletion *orders*, or the converse, stated. The refusal probability itself is classical: it is the tail of the unsuccessful-search cost, whose exact finite-size generating function and Poisson limit are due to Knuth's run-length analysis [24] and to Janson and Viola [22]; we claim only an elementary derivation of an explicit finite-size sum, the quantitative gap to the Chernoff-type bound, the design rule, and the validation against an implementation. The adversary results are model-based (ideal hash, idealised oracle), not timing measurements. The proofs are pen-and-paper and are backed by exhaustive and statistical checks, not by a proof assistant. Section 8 lists what the evidence does not cover.

Outline. Section 2 fixes the model. Sections 3–6 give the four groups of results. Section 7 places them in the literature, Section 8 states limitations, and Section 9 concludes. Appendix A proves Theorem 7; Appendix B explains how to regenerate every number.

2 Model

Tables and schemes. A table has $m \geq 2$ cells, indexed by $\mathbb{Z}_m$. Its state T maps each cell to a name or to $\perp$ (empty); $\mathrm{Occ}(T) \subseteq \mathbb{Z}_m$ is the set of non-empty cells. Every name x has a *probe sequence* $\pi_x = (\pi_x(1), \pi_x(2), \dots)$ of cells, determined by the hash function (and its key). For linear probing, $\pi_x(i) = h(x) + i - 1 \bmod m$ for a *home* $h(x) \in \mathbb{Z}_m$.

Definition 1 (Non-reordering scheme)**.** An open-addressing scheme is *non-reordering* if an insertion of x into T writes x into one cell $\rho(\mathrm{Occ}(T), \pi_x) \in \{\pi_x(i)\} \setminus \mathrm{Occ}(T)$ — or refuses, if the rule returns $\perp$ — and changes no other cell. The rule ρ may depend on the table only through the occupancy set (so control state that is a function of the occupancy, such as the number of occupied cells or per-region counters, is allowed).

This covers linear, quadratic and double hashing, uniform probing, and non-greedy rules that decide from occupancy counts alone (funnel and elastic hashing of [12] are of this kind, as checked in the proof of Theorem 6). It excludes cuckoo and hopscotch hashing [19, 32] and Robin Hood hashing [8], which move keys.

Stack discipline. The operations are PUSH(x), which inserts a name that is not live, and POP, which removes the most recently pushed live name by setting its cell to $\perp$. A scope mark is a position in the live sequence, and leaving the scope is the sequence of pops back to that position. A push that the rule refuses (buffer exhausted, probe limit exceeded) changes nothing and the name does not become live. The *live sequence* is the list of live names in the order pushed. For a sequence $a = (a_1, \dots, a_t)$ of distinct names, $\mathrm{Ins}(a)$ denotes the table obtained by inserting $a_1, \dots, a_t$ in this order into the empty table.

Hash model. In the *random-function model*, the probe sequences of distinct names are independent and identically distributed; for linear probing the homes are independent and uniform on $\mathbb{Z}_m$. An operation sequence is *oblivious* if it does not depend on the hash function. A keyed hash is modelled as a random function drawn from the key; Section 5 says what this assumption does and does not give against an adversary.

Mapping to STRATA. In STRATA the index is a ring of m words (m a power of two) probed linearly; a name that hides a visible one needs no index slot, so the pushes of the model are the *first* declarations of a name; shadowing declarations are recorded on the undo stack only. Exiting a scope pops the undo stack and, for each first declaration, sets one index slot to zero. In the growable mode a rebuild re-inserts the names oldest first, which re-establishes Ins of the live sequence for the larger index.

3 Exact Undo and Its Converse

Theorem 2 (Undo)**.** *Let ρ be a non-reordering rule. After any sequence of pushes and pops, with refused pushes leaving the state unchanged, the table equals* $\mathrm{Ins}(a)$*, where a is the live sequence.*

Proof. Induction on the number of operations. The empty table is $\mathrm{Ins}(())$. A refused push changes neither the table nor a. If a push of x succeeds, the table before it is $\mathrm{Ins}(a)$ by hypothesis; the push writes x into $\rho(\mathrm{Occ}(\mathrm{Ins}(a)), \pi_x)$, which is $\mathrm{Ins}(a \cdot x)$ by definition. For a pop, $a = u \cdot x$ and the table is $\mathrm{Ins}(u \cdot x)$, which is $\mathrm{Ins}(u)$ with x additionally in the cell that the insertion of x chose, because insertion changes exactly one cell and that cell was empty in $\mathrm{Ins}(u)$. Clearing it gives $\mathrm{Ins}(u)$. □

Two features of the proof matter. It does not use the lookup rule, so *any* lookup procedure that is correct for insertion-only tables — stopping at the first empty cell for greedy schemes, level-by-level searches for non-greedy ones — stays correct under stack discipline. And it does not use randomness, so it also holds for adversarial homes.

Corollary 3 (Marginals)**.** *In the random-function model, for an oblivious operation sequence, at every moment up to the first refusal the law of the table is the law of the insertion-only table of the live sequence. In particular, for linear probing, Knuth's expected probe counts [24], the distribution of displacements [21, 37], and the refusal law of Section 4 hold at every moment, for the number n of live keys at that moment.*

This concerns the *state* of the table and the cost of the *next* operation; cumulative or amortised costs over a whole schedule do not transfer. Successive moments are correlated (a name that is popped and pushed again returns to the same cell if the live sequence below it is unchanged), so Corollary 3 is a statement about marginals; Section 4 uses it with a union bound.

3.1 Why stack order is exactly right

For linear probing, a *lookup* of x scans from $h(x)$ and stops at x or at an empty cell. Call a deletion order σ of n keys $x_1, \ldots, x_n$ (inserted in this order into $\mathbb{Z}_m$, $2 \le n \le m$) *clear-safe for the homes h* if after every prefix of σ has been cleared, the lookup of every remaining key still finds it.

For a table built by $\mathrm{Ins}(x_1, \ldots, x_n)$ let $c(x)$ be the cell of x and say that y *depends on* x, written $y \lhd x$, if $c(x)$ lies strictly before $c(y)$ on the probe path of y (for linear probing: between $h(y)$ and $c(y)$, cyclically). A key y that depends on x was inserted after x, since $c(x)$ was occupied when y was inserted.

Theorem 4 (Exactness)**.** *Fix the homes of $x_1, \ldots, x_n$ and a deletion order σ of them.*

(a) σ is clear-safe for these homes if and only if y is deleted before x whenever $y \lhd x$, i.e. if and only if σ is a linear extension of the dependency relation $\lhd$.

(b) σ is clear-safe for every assignment of homes if and only if $\sigma = (x_n, x_{n-1}, \ldots, x_1)$, the reverse insertion order.

Proof. (a) Clearing never moves a key, so while no insertion happens each cell still holds the key it held, or is empty. The lookup of a remaining key y follows its probe path and stops at the first empty cell; the cells of that path before $c(y)$ were occupied when y was inserted, by keys x with $y \lhd x$, and $c(y)$ holds y. So y is found if and only if none of those keys has been cleared. Thus every lookup succeeds after every prefix of σ if and only if no x is cleared while a $y \lhd x$ remains.

(b) ($\Leftarrow$) $y \lhd x$ implies that y is newer than x, so reverse insertion order is a linear extension of $\lhd$ for every assignment, and (a) applies. ($\Rightarrow$) Let $i < j$. Give $x_1, \ldots, x_{j-1}$ distinct homes (possible since $j - 1 < m$), so each sits in its own home cell, and give x_j the home of x_i. Then $c(x_i) = h(x_i)$ is the first cell of the probe path of x_j and is not $c(x_j)$, so $x_j \lhd x_i$. By (a), a σ that is clear-safe for this assignment deletes x_j before x_i. If σ is clear-safe for every assignment, this holds for every pair $i < j$, which means σ is the reverse insertion order. □

Remark 1. Part (a) uses only that a lookup follows the probe sequence of the key and stops at the first empty cell, so it holds verbatim for every greedy probing scheme (linear, quadratic, double hashing, uniform probing), with $\lhd$ defined through the probe sequences. Part (b) needs more than that: the probe sequence of a key must be able to start at any cell and must contain a second distinct cell (a scheme with one probe per key has no dependencies, and every order is safe), and "every assignment" must mean every assignment in which all keys are inserted (quadratic probing $h + i^2$ modulo a power of two reaches only half of the cells and refuses some assignments). Under these conditions the pair construction works with x_j homed at the first cell of x_i and the earlier keys placed in cells outside the first two probes of x_j. We checked (b) exhaustively for linear probing and, in addition, for quadratic probing ($m \in \{4, 8\}$, $n \leq 4$), and part (a) for linear probing on every assignment and every deletion order with $m \leq 5$ (397 562 pairs).

For a single deletion the condition of part (a) is the test of Sanders [34, Sect. 2] for when a deleted element must become a tombstone; we extend it to sequences of deletions and prove that stack order is the only universally safe order. Part (a) says exactly how much freedom a particular table leaves: the safe orders are the linear extensions of $\lhd$, and stack order is always one of them. A key that nothing depends on can be removed at once; Proposition 5 gives the law of the number of keys that depend on a key.

Theorem 4 explains why every deletion method for open addressing that supports arbitrary order needs more than clearing: tombstones, or backward shifting (Knuth's Algorithm R) [23, 24], whose long-run behaviour is analysed in [5]. It also explains why STRATA cannot simply swap in a relocating deletion: its undo entries store index slots, and relocation would invalidate them. The next proposition quantifies the damage of leaving the discipline.

Proposition 5 (Orphans)**.** *In a linear-probing table, let δ_y be the displacement of live key y (the number of cells from its home up to, but excluding, its own cell, cyclically; one less than its probe count) and, for a live key x, let* $\operatorname{orph}(x) = \#\{y : y \lhd x\}$ *be the number of other live keys whose lookup fails after the cell of x is cleared. Write $a_j = e^{-\alpha}\alpha^j/j!$.*

(a) $\operatorname{orph}(x)$ *depends only on the cell of x, not on which key occupies it or on the insertion order: it equals $Y_c - 1$, where Y_c is the number of live keys whose path from home to own cell contains $c = c(x)$, and $Y_c = (Y_{c-1} - 1)^+ + A_c$, with A_c the number of keys homed at c. Moreover $\sum_x \operatorname{orph}(x) = \sum_y \delta_y$, so clearing a uniformly random live key orphans on average the mean displacement of a key; the newest key orphans none.*

(b) For $n/m \to \alpha \in (0, 1)$ with independent uniform homes, the number of orphans of a uniformly random live key converges in law to $Y - 1$ conditioned on $Y \geq 1$, where Y has the stationary law (p_j) of the chain $Y \mapsto (Y - 1)^+ + A$ with $A \sim \operatorname{Poisson}(\alpha)$ (a discrete-time

M/D/1 queue):

$$\mathbb{P}(\mathrm{orph}=j)\ \to\ p_{j+1}/\alpha, \qquad p_0=1-\alpha, \quad p_1=(1-\alpha)(e^{\alpha}-1),$$
$$p_{j+1}=\frac{p_j-p_0a_j-\sum_{i=1}^{j}p_ia_{j-i+1}}{a_0} \quad (j\ge 1).$$

In particular the fraction of live keys that nothing depends on, and that can therefore be removed immediately in any order, is $(1-\alpha)(e^{\alpha}-1)/\alpha$ *(*0.649 *at* $\alpha=\frac{1}{2}$, 0.372 *at* $\frac{3}{4}$*), and the mean is* $\alpha/(2(1-\alpha))$ *(for example* 1.5 *at* $\alpha=3/4$*).*

(c) For finite m *and* $n=m-k$, $k\ge 1$, *the expected number of live keys with exactly* j *orphans is*

$$k\sum_{g=2}^{n+1}\frac{Q_{k-1}(m-g)}{Q_k(m)}\sum_{u=1}^{g-1}F_u(j)\,Q_{j+1}(g-u),$$

where $F_0(0)=1$ *and* $F_{u+1}(h')=\sum_{y\ge 0,\ h=h'+1-y\ge 0}F_u(h)/y!$ *(with* Q_j *as in Section 4); dividing by* n *gives the law for a random live key.*

The limit law in (b) is the distribution of the Robin Hood displacement of linear probing found by Janson [21, Thm. 8.1, Eq. (8.4)], who also relates the underlying count to the same M/D/1 queue; what we add is its reading as the number of keys orphaned by a deletion (Theorem 4(a)), the exact finite-size form (c), and the order-independence in (a). Table 1 shows the limit law. At load 3/4 only about a third of the live keys can be removed out of order without harm (the finite-size law of part (c) gives 0.387 at $m=128$ and 0.380 at $m=256$, against the limit 0.372). The count of dependents has a geometric tail: $\mathbb{P}(\mathrm{orph}=j)$ decays like z_*^{-j}, where $z_*>1$ solves $z=e^{\alpha(z-1)}$ ($z_*^{-1}\approx 0.58$ at $\alpha=3/4$, matching the successive ratios in the table); it is slow as $\alpha\to 1$.

Proof. (a) A key y is orphaned by clearing x iff $y\lhd x$ (Theorem 4(a)), i.e. iff the cell c of x lies on the path of y strictly before y's own cell. The live keys whose path contains c are those homed at c and those whose path contains $c-1$ and whose cell is not $c-1$; if $c-1$ is occupied exactly one key whose path contains it has its cell there, and if it is empty no path contains it. This is the recursion for Y_c; c is occupied iff $Y_c\ge 1$, the key at c is one of the Y_c, and the other Y_c-1 are the keys depending on it. Hence $\mathrm{orph}(x)=Y_{c(x)}-1$. Summing over occupied cells, $\sum_x\mathrm{orph}(x)=\sum_c(Y_c-1)^+$, which is the number of pairs (cell, key whose path passes through it strictly before the key's own cell), i.e. $\sum_y\delta_y$. The total displacement does not depend on the insertion order (it is a function of the home counts alone [24]); its mean over independent uniform homes is $n(C_n-1)$, where C_n is Knuth's expected cost of a successful search, and $C_n-1\to\frac{1}{2}(\frac{1}{1-\alpha}-1)=\frac{\alpha}{2(1-\alpha)}$. That the newest key has no dependents is Theorem 4.

(b) Consider the ring with $n<m$ keys. By Appendix A, Step 3, the block containing cell 0 has more than w cells with probability $1-\frac{k}{m}\sum_{g\le w}g\pi_g$, because $\sum_g g\pi_g=m/k$; each π_g converges to $\mathrm{Borel}(g;\alpha)$ (Corollary 8(b)) and $m/k\to 1/(1-\alpha)=\sum_g g\,\mathrm{Borel}(g;\alpha)$, so this probability tends to $(1-\alpha)\sum_{g>w}g\,\mathrm{Borel}(g;\alpha)$; given $\varepsilon>0$ choose w so that this is below $\varepsilon/2$, and it is below ε for all large m. Run the recursion $Y\mapsto(Y-1)^+ + A$ from the value 0 at cell $-w$; the result $Y_0^{(w)}$ is a function of the home counts of cells $-w+1,\dots,0$ only, it never exceeds the true Y_0 (the recursion is monotone in its initial value), and it equals Y_0 whenever some cell among $-w,\dots,0$ is empty, because the true value is then 0 there too. By the preceding paragraph the probability that no cell in a window of w cells is empty is at most ε for large m. The joint law of these home counts converges to independent $\mathrm{Poisson}(\alpha)$ variables, so $Y_0^{(w)}$ converges in law to the w-step value of the chain started from 0, which converges to its stationary law (p_j) as $w\to\infty$ (the chain is positive recurrent since $\alpha<1$). Hence Y_0 converges in law to (p_j). A uniformly random live key sits at a uniformly random occupied cell, so $\mathbb{P}(\mathrm{orph}=j)=\mathbb{P}(Y_0=j+1)/\mathbb{P}(Y_0\ge 1)\to p_{j+1}/\alpha$. The balance equations

Table 1: Limit law of the number of orphans of a uniformly random live key, $\mathbb{P}(\text{orph} = j)$ for $j = 0, \ldots, 4$, and its mean $\alpha/(2(1-\alpha))$. The first column is the fraction of keys that nothing depends on.

α	j=0	1	2	3	4	mean
0.25	0.852	0.131	0.015	0.002	0.000	0.167
0.5	0.649	0.245	0.076	0.022	0.006	0.500
0.75	0.372	0.259	0.156	0.090	0.052	1.500
0.9	0.162	0.153	0.128	0.104	0.085	4.500

$p_j = p_0 a_j + \sum_{i=1}^{j+1} p_i a_{j-i+1}$ at $j = 0$ give $p_1 = p_0(1/a_0 - 1)$ and, solved for p_{j+1}, the recursion; $p_0 = 1 - \alpha$ because the stationary fraction of occupied cells is α. The mean follows from the classical stationary mean $\mathbb{E}[Y] = \alpha(2-\alpha)/(2(1-\alpha))$ of this queue: $(\mathbb{E}[Y]-\alpha)/\alpha = \alpha/(2(1-\alpha))$; convergence of the mean of the finite-m law is part (a).

(c) Cut the ring at its empty cells as in Appendix A. In a block of g cells with patterns $y_1, \ldots, y_{g-1}, y_g = 0$, the key at the cell $u+1$ ($1 \le u \le g-1$) has orph $= Z_u = \sum_{t \le u}(y_t - 1)$, because $Z_u = Y_c - 1$ along the walk of Step 2. The weight of the part of the walk up to step u with $Z_u = h$ and $Z_t \ge 0$ is $F_u(h)$ (the recursion records exactly the steps $y - 1 \ge -1$ that stay non-negative), and the rest is a first passage from h to -1 in $g - u$ steps, with weight $Q_{h+1}(g-u)$ by the Borel–Tanner identity. By Step 3, the weight of the other $k-1$ blocks is $Q_{k-1}(m-g)$ against the normalisation $Q_k(m)$, and the factor k comes from the normalisation $\frac{k}{m}\sum_g g\pi_g \cdot \frac{1}{g} \cdot g$ of Step 3 (a uniform cell lies in a block of size g with probability $\frac{k}{m} g\pi_g$, at a uniform position). □

3.2 Consequence: a separation by deletion order

The best insertion-only bounds for open addressing without reordering are due to Farach-Colton, Krapivin and Kuszmaul [12]: a greedy scheme (funnel hashing) with worst-case expected probe complexity $O(\log^2 \delta^{-1})$ and amortised $O(\log \delta^{-1})$ for the insertions up to load $1 - \delta$, and a non-greedy scheme (elastic hashing) with worst-case expected $O(\log \delta^{-1})$ and amortised $O(1)$. For insertions *and* deletions over an unbounded horizon the same paper states that even linear probing and uniform probing resist analysis, and that the optimal amortised expected probe complexity is $\delta^{-\Omega(1)}$, citing Theorem 3 of [6]. We checked that statement in the primary source: Theorem 3 there lower-bounds the size of variable-size tiny pointers by $\Omega(\log \delta^{-1})$ bits at load $1 - \delta$ with $\delta < 1/4$, for a workload that alternates insertions of random elements with deletions of random present elements. An open-addressing scheme induces such pointers by encoding the probe index i in $O(\log i)$ bits, so $\mathbb{E}[\log i] = \Omega(\log \delta^{-1})$ and, by Jensen, $\mathbb{E}[i] \ge \delta^{-\Omega(1)}$; this is the reduction of [12], which we have not re-proved.

For a scheme $\mathcal{S}$ that is non-reordering (Definition 1) with independent probe sequences, write q_j for the expected probe index of the j-th insertion into an empty table (the index of the probe that finds the cell used), $1 \le j \le (1-\delta)m$, and call a workload *oblivious stack-disciplined* if it is a sequence of pushes of fresh names and pops that does not depend on the hash functions and keeps at most $(1-\delta)m$ names live. Its *worst-case cost* is the largest expected probe index of a single push, and its *amortised cost* is the expected probe index averaged over its pushes, each maximised over such workloads.

Theorem 6 (Probe complexity under stack discipline)**.** *Let $\mathcal{S}$ be non-reordering with independent probe sequences.*

(i) In every oblivious stack-disciplined workload, up to its first refused push, the expected probe index of a push that finds $j-1$ live names is exactly q_j (for a scheme that does not refuse below load $1-\delta$, this covers the whole workload). Hence the worst-case cost

and the amortised cost of $\mathcal{S}$ over such workloads are both $\max_{j\le(1-\delta)m} q_j$, the worst-case expected cost of the insertion-only table.

(ii) Consequently, under the conditions on δ of [12]: elastic hashing has worst-case and amortised cost $O(\log \delta^{-1})$ and funnel hashing $O(\log^2 \delta^{-1})$ on every oblivious stack-disciplined workload; every non-reordering scheme has worst-case and amortised cost $\Omega(\log \delta^{-1})$, and every greedy scheme $\Omega(\log^2 \delta^{-1})$. So the optimum is $\Theta(\log \delta^{-1})$ (non-greedy) and $\Theta(\log^2 \delta^{-1})$ (greedy).

(iii) For comparison, on insertion-only workloads the amortised optimum is $O(1)$ (elastic hashing), and on workloads that alternate insertions of random names with deletions of random live names the optimum is $\delta^{-\Omega(1)}$ [6, 12].

Proof. (i) Let a push find the live sequence a of $j-1$ names. By Theorem 2 the table is $\mathrm{Ins}(a)$, a function of the probe sequences of the names of a alone, and the pushed name is fresh, so its probe sequence is independent of them. The cell it uses, and the index of the probe that finds it, are functions of $\mathrm{Occ}(\mathrm{Ins}(a))$ and of its probe sequence (Definition 1), exactly as in the j-th insertion of an insertion-only run, whose live sequence is any $j-1$ names; so the expectation is q_j. The workload is oblivious, so a does not depend on the hashes. Every push of a stack-disciplined workload is therefore bounded by $\max_j q_j$, which gives the upper bound for both costs. For the lower bound let j^* attain the maximum (it depends on $\mathcal{S}$, m and δ but not on the hashes), push j^*-1 names, then repeat N times: push a fresh name, pop it. By Theorem 2 each of the N repeated pushes finds the table $\mathrm{Ins}(a)$ of the same j^*-1 names and costs q_{j^*} in expectation, so the average tends to q_{j^*} as $N\to\infty$.

(ii) The upper bounds are Theorems 1 and 2 of [12] combined with (i); the lower bounds are its Theorem 5 (some insertion has expected probe index $\Omega(\log \delta^{-1})$ in any non-reordering scheme that reaches load $1-\delta$) and Theorem 4 (the last insertion at load $1-\delta$ costs $\Omega(\log^2 \delta^{-1})$ in any greedy scheme), again combined with (i). Funnel hashing and elastic hashing satisfy Definition 1: funnel hashing is greedy with a fixed probe sequence per name (the bucket of each level, level after level, then the special array), so it takes the first empty cell of the sequence; in elastic hashing the batch, the fullness ϵ_1, ϵ_2 of the two arrays involved and whether a cell is free are functions of the occupancy counts, provided the implementation keeps its counters as functions of the occupancy (decrementing them when a name is popped), which we assume; then the state after any sequence of pushes and pops is a function of the occupancy, and Theorem 2 applies by induction on the operations. The elastic claim is conditional on this assumption, which [12] does not discuss since it considers insertions only.

(iii) Quoted: the first part is Theorem 1 of [12]; the second is the statement of [12] derived from Theorem 3 of [6] described above, whose reduction we have not re-proved. □

Parts (i) and (ii) give a complete and tight answer for the stack-disciplined case, and place it strictly between the two known regimes: the amortised optimum is $O(1)$ for insertions only, $\Theta(\log \delta^{-1})$ under stack discipline (the oscillating workload makes every push as expensive as the worst insertion; no amortisation survives), and $\delta^{-\Omega(1)}$ for random deletions. Deletion *order* therefore moves the achievable probe complexity of open addressing without reordering from polynomial to polylogarithmic in δ^{-1}, and a workload that never reorders but frees names in the order of their creation, as every scoped symbol table does, falls on the good side. The statements concern expected probe indices in the random-function model against oblivious workloads; they say nothing about worst-case schedules chosen with knowledge of the hashes.

Table 2 lists the consequence for the four schemes with known bounds, at load $1-\delta$. For linear probing the worst-case insertion costs $\frac{1}{2}(1+\delta^{-2})$ probes in expectation (Knuth) while the cost averaged over a fill is $\Theta(\delta^{-1})$; by Theorem 6(i) stack discipline at the top load costs the full $\Theta(\delta^{-2})$ on every push, so the price of stack-ordered deletion for linear probing is a factor δ^{-1} in amortised cost, against a factor $\log \delta^{-1}$ for elastic hashing and none for the worst case.

Table 2: Expected probe index at load $1-\delta$ (orders of magnitude; $\delta \to 0$). Insertion-only: worst single insertion / averaged over the fill. Stack discipline: worst push = averaged over the worst oblivious schedule (Theorem 6(i)). Linear and uniform probing: Knuth [24], Yao [38]; funnel and elastic hashing: [12].

scheme	insertion-only: worst / amortised	stack: worst = amortised
linear probing	δ^{-2} / δ^{-1}	δ^{-2}
uniform probing	δ^{-1} / $\log \delta^{-1}$	δ^{-1}
funnel hashing (greedy)	$\log^2 \delta^{-1}$ / $\log \delta^{-1}$	$\log^2 \delta^{-1}$
elastic hashing (non-greedy)	$\log \delta^{-1}$ / 1	$\log \delta^{-1}$

4 An Exact Refusal Law

A bounded-work table fixes a *probe limit* L: an insertion examines at most L cells from its home and is refused (nothing changed) if all are occupied. Because a stored key never moves and clearing only empties cells, every stored key lies within L cells of its home, so lookups also stop after L cells; the work of a call is bounded by a constant independent of the contents. The question is how often an *honest* key is refused.

By Corollary 3, the refusal probability at a moment with n live keys is that of an insertion-only linear-probing table with n keys: the probability that an independent new key with uniform home finds its first L cells occupied. Equivalently, it is the probability that the cost of an unsuccessful search exceeds L, a quantity whose distribution has been studied extensively [14, 20, 24]. Janson and Viola give the exact generating function for finite m, n and, in the infinite Poisson model, $\mathbb{P}(U = k) = (1-\alpha)\mathbb{P}(B > k)$ for the block length B [22, Thm. 11.1, Eq. (11.4)]; Knuth's run-length probabilities [24] give the same finite-size values by Abel-binomial counting.

Notation. For $g \geq 1$ let $B(g) = g^{g-1}/g!$, and for $j \geq 1$, $s \geq j$ let $Q_j(s) = \frac{j}{s}\, s^{s-j}/(s-j)!$ and $Q_0(s) = \mathbf{1}[s = 0]$. By the Borel–Tanner identity [36], $Q_j(s) = \sum_{g_1+\cdots+g_j=s} \prod_i B(g_i)$.

Theorem 7 (Exact refusal law)**.** *Let $m \geq 2$ cells in a ring hold $n = m - k$ keys, $k \geq 1$, with independent uniform homes under linear probing, and let a new key have an independent uniform home. The probability $R_m(n, L)$ that its first L cells are all occupied is*

$$R_m(n, L) \;=\; \frac{k}{m} \sum_{g=L+1}^{n+1} (g - L)\, \pi_g, \qquad \pi_g = \frac{B(g)\, Q_{k-1}(m-g)}{Q_k(m)}. \tag{1}$$

Here $\pi_g = \mathbb{P}(\gamma_1 = g \mid \gamma_1 + \cdots + \gamma_k = m)$ for i.i.d. Borel variables γ_i, and does not depend on the Borel parameter. For $k = 1$, $R_m(m-1, L) = (m-L)^+/m$.

Remark 2. Theorem 7 computes a known quantity: $R_m(n, L)$ is the tail of the unsuccessful-search cost, and Corollary 8(b) is the form $(1-\alpha)\mathbb{E}[(B-L)^+]$ of [22, Eq. (11.4)] with B Borel distributed. What the proof below adds is a short derivation that uses only the cycle lemma and the Borel–Tanner identity and yields the explicit sum (1); it was cross-checked against enumeration and against the implementation (Section 4).

The proof (Appendix A) cuts the ring at its k empty cells; each empty cell and the run of occupied cells after it form a block whose size has the Borel distribution (the total progeny of a Poisson branching process, by the cycle lemma [11]), and the blocks, conditioned on filling the ring, are exchangeable.

Corollary 8 (Limit, monotonicity, asymptotics)**.** *(a) $R_m(n, L)$ is nondecreasing in n, so the value at the maximum load bounds every lower load. (b) If $n/m \to \alpha \in (0, 1)$ then*

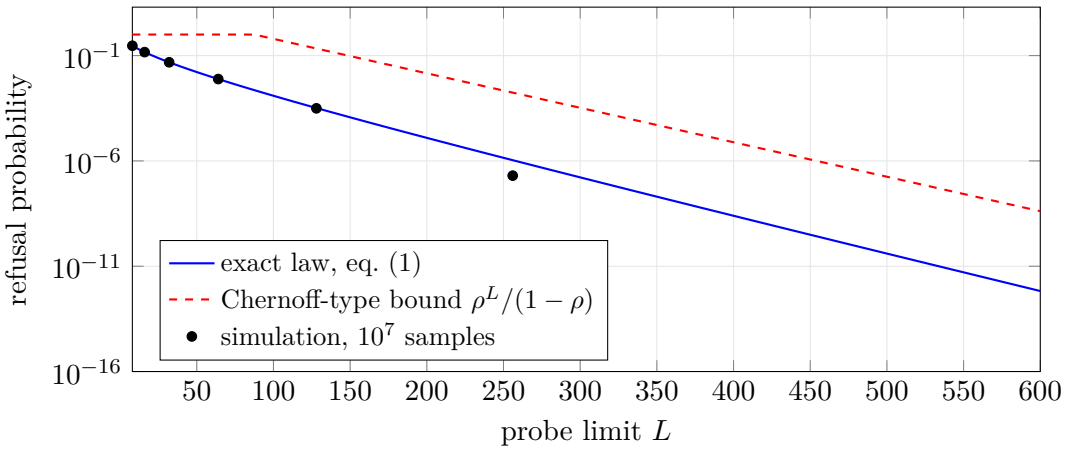


Figure 1: Probability that an insertion needs more than L probes, $m = 65\,536$ cells, load $3/4$. The exact law agrees with the stored simulation of the repository where the latter observes events; the Chernoff-type bound is looser by a factor that grows with L.

$R_m(n, L) \to R_\infty(\alpha, L) = (1-\alpha)\sum_{g>L}(g-L)\,\mathrm{Borel}(g;\alpha)$, *with* $\mathrm{Borel}(g;\alpha) = e^{-\alpha g}(\alpha g)^{g-1}/g!$. *(c) With* $\rho = \alpha e^{1-\alpha} < 1$,

$$R_\infty(\alpha, L) \;\sim\; \frac{(1-\alpha)\,\rho^{L+1}}{\alpha\sqrt{2\pi}\,(1-\rho)^2\,L^{3/2}} \qquad (L \to \infty).$$

Proof. (a) The occupied set of linear probing does not depend on the insertion order [25]. Insert the $(n+1)$-st key last: the occupied set with $n+1$ keys is the occupied set with n keys plus one cell, and the event "the first L cells are occupied" is increasing in the occupied set. (b) Write π_g via the explicit Q_j: $\pi_g = B(g)\,\frac{k-1}{k}\,\frac{m}{m-g}\,\frac{(m-g)^{n-g+1}\,n!}{m^n\,(n-g+1)!}$, and for fixed g with $n = \alpha m$ this tends to $B(g)\alpha^{g-1}e^{-\alpha g} = \mathrm{Borel}(g;\alpha)$. Since $(g-L)^+ = g - \min(g, L)$, $\sum_g \pi_g = 1$ and $\sum_g g\pi_g = m/k$ (Appendix A, Step 3),

$$R_m(n, L) = \frac{k}{m}\Big[\frac{m}{k} - \sum_{g\le L} g\pi_g - L\Big(1 - \sum_{g\le L}\pi_g\Big)\Big],$$

which involves only the finitely many π_g with $g \le L$. Each tends to its Borel limit and $m/k \to 1/(1-\alpha) = \sum_g g\,\mathrm{Borel}(g;\alpha)$, so the right side converges to the same expression with $\mathrm{Borel}(g;\alpha)$ in place of π_g, which is $(1-\alpha)\sum_{g>L}(g-L)\,\mathrm{Borel}(g;\alpha)$. (No uniform-integrability argument is needed.) (c) Stirling's formula gives $\mathrm{Borel}(g;\alpha) \sim \rho^g/(\alpha\sqrt{2\pi}\,g^{3/2})$; then $\sum_{j\ge1} j\,\mathrm{Borel}(L+j;\alpha) \sim \rho^L(\alpha\sqrt{2\pi}L^{3/2})^{-1}\sum_j j\rho^j$ by dominated convergence. □

Comparison with the Chernoff-type bound. The repository's earlier analysis of the probe limit [35] used the interval argument of linear probing with a Chernoff bound and obtained $\mathbb{P}(\text{refusal}) \le \rho^L/(1-\rho)$. The bound is valid. Corollary 8(c) shows that it overstates the truth by a factor $\sim \alpha\sqrt{2\pi}\,(1-\rho)L^{3/2}/((1-\alpha)\rho) \to \infty$: a union over all run lengths $\ge L$ pays a factor $1/(1-\rho)$ for events that overlap heavily, and the Chernoff estimate omits the polynomial factor $L^{-3/2}$ of the Borel tail. At load $3/4$ and $m = 65\,536$ the exact value at $L = 512$ is 2.44×10^{-11} against the bound's 1.13×10^{-7}, a factor of $4\,637$; at $L = 64$ the factor is 315 (Figure 1).

Design rule. The smallest limit that keeps the per-insertion refusal probability below ε is read off the exact law (Table 4, Figure 2). Two points are worth stating. For large tables the exact limit is about 57% of the Chernoff-derived one at load $3/4$ (258 against 455 for $\varepsilon = 10^{-6}$). And small tables need much less: the finite-size law is smaller than its Poisson limit at the loads tabulated, so a 256-cell index needs only $L = 121$.

Other probing rules. For uniform probing, where the probe sequence of a name is a uniformly random sequence of distinct cells independent of the table, the probability that the first L probes all hit occupied cells is exactly $\binom{n}{L}/\binom{m}{L} = \prod_{i<L} \frac{n-i}{m-i} \le \alpha^L$ *whatever the occupied set*; by Theorem 2 it therefore holds at every moment under stack discipline. For $\varepsilon = 10^{-6}$ and $m = 2^{16}$ it gives $L = 20$ at load $1/2$ and $L = 49$ at load $3/4$, against 52 and 258 for linear probing (about one fifth of the work per call), at the price of the cache locality that makes linear probing attractive; double hashing and quadratic probing approximate uniform probing but are not covered by this exact value. A caller who wants a failure probability over a whole compilation of N insertions uses $\varepsilon = \varepsilon'/N$, by the following union bound.

Proposition 9 (Run of a compilation)**.** *Under the random-function model with an oblivious sequence of N pushes of names distinct from the live ones, and at most $n_{\max}$ live keys, the probability that some push is refused is at most $N\, R_m(n_{\max}, L)$.*

Proof. Until the first refusal the process coincides with the unlimited process. At each push the new name has an independent uniform home, independent of the live names' homes, and the table is the insertion-only table of the live sequence (Corollary 3); the refusal probability is $R_m(n_t, L) \le R_m(n_{\max}, L)$ by Corollary 8(a). Union bound over the pushes. (The same argument gives the sharper bound $\sum_t R_m(n_t, L)$, with n_t the live count at push t.) □

The factor N is pessimistic when the live count rises gradually, because the early pushes are far less likely to be refused than the last; for the fill of a table to a given load the whole-run probability can be computed exactly.

Theorem 10 (Refusal-free fill)**.** *Let $n < m$, $k = m - n$, and let n pushes of distinct fresh names (independent uniform homes) be made into an empty linear-probing ring of m cells with probe limit L. The probability that none is refused is*

$$P_{\text{fill}}(m, n, L) \;=\; \frac{n!}{m^n} \cdot \frac{m}{k}\, [z^n]\, \Phi_L(z)^k, \qquad \Phi_L(z) = \sum_{j \ge 0} A_L(j) \frac{z^j}{j!},$$

where $A_L(0) = 1$ and $A_L(j) = \sum_{s=1}^{j} \binom{j-1}{s-1} \min(L, s)\, A_L(s-1) A_L(j-s)$. Equivalently $\Phi_L' = \Phi_L\,(L\Phi_L - P_L)$ with $P_L(z) = \sum_{i=0}^{L-2} (L-1-i) \frac{(i+1)^{i-1}}{i!} z^i$, whose solution is

$$\Phi_L(z) = \frac{e^{-Q_L(z)}}{1 - L \int_0^z e^{-Q_L(t)}\, dt}, \qquad Q_L(z) = \int_0^z P_L.$$

Moreover $1 - P_{\text{fill}} \le \sum_{t=0}^{n-1} R_m(t, L)$.

Proof. Until the first refusal the process is the unlimited one, so the fill is refusal-free iff every key has displacement less than L when it is inserted. Cut the ring at its k empty cells as in Appendix A: the set of empty cells depends only on the multiset of homes, and keys of different blocks never interact. A block with j filled cells receives exactly the keys homed in them; its time-ordered subsequence of homes is a sequence in $\{1, \dots, j\}^j$ that fills the block with all displacements below L; let $A_L(j)$ be the number of such sequences. Consider the last of these keys and let it end in cell s. Cell s was empty until then, so every earlier key stayed left of s (homes in $1..s-1$) or right of it (homes in $s+1..j$); the two groups are independent valid sequences of lengths $s-1$ and $j-s$, interleaved in time in $\binom{j-1}{s-1}$ ways, and the last key may have any home in $\{s-L+1, \dots, s\} \cap \{1, \dots, s\}$, $\min(L, s)$ choices, since the cells between its home and s are occupied. This is the recursion. For the ring, choose the set E of empty cells; the keys are divided among the blocks in $n! / \prod_i (g_i - 1)!$ ways (g_i the block sizes) and each block contributes $A_L(g_i - 1)$; summing over E gives $\frac{m}{k}$ times the sum over ordered compositions, $n! \frac{m}{k} [z^m] (z\Phi_L)^k$, and dividing by m^n gives the formula. For the differential equation, divide the recursion by

Table 3: Smallest probe limit L for which a fill to load 3/4 is refusal-free with probability at least $1-\varepsilon'$: exact (Theorem 10), by the sum $\Lambda=\sum_t R_t$, and by $N\,R_{\max}$ with $N=n$; last column: true failure probability divided by Λ at the exact limit.

m	ε'	exact	$\sum_t R_t$	$NR_{\max}$	$\mathbb{P}/\Lambda$
256	0.01	62	65	94	0.71
256	0.0001	98	99	122	0.86
256	1e-06	123	124	141	0.93
1024	0.01	98	104	162	0.63
1024	0.0001	165	169	227	0.74
1024	1e-06	227	230	284	0.82
4096	0.01	131	139	217	0.61
4096	0.0001	218	224	311	0.70
4096	1e-06	306	312	402	0.76

Table 4: Smallest probe limit L with per-insertion refusal probability at most ε: exact finite-size law for index sizes m, its Poisson limit ($m\to\infty$), and the limit that the Chernoff-type bound would require.

load	ε	$m{=}2^8$	2^{10}	2^{12}	2^{14}	2^{16}	$m\to\infty$	Chernoff
3/8	10^{-6}	25	28	29	29	29	29	43
3/8	10^{-9}	37	44	46	47	47	47	62
1/2	10^{-6}	42	49	51	52	52	52	81
1/2	10^{-9}	59	77	83	84	85	85	117
5/8	10^{-6}	70	93	101	104	104	104	171
5/8	10^{-9}	95	142	162	168	170	170	244
3/4	10^{-6}	121	201	242	255	258	260	455
3/4	10^{-9}	148	289	380	413	422	426	638

$(j-1)!$: $\Phi_L'=\Phi_L\sum_{i\ge0}\min(L,i+1)\frac{A_L(i)}{i!}z^i$ and $\min(L,i+1)=L-\max(0,L-1-i)$, with $A_L(i)=(i+1)^{i-1}$ for $i\le L-2$ (all parking functions of length i are valid). This is a Bernoulli equation; $u=1/\Phi_L$ satisfies $u'=P_Lu-L$, $u(0)=1$, which integrates to the stated form. The inequality is the union bound of Proposition 9 with $n_t=t$. □

The numbers $A_L(j)$ are those of the ℓ-interval parking functions with $\ell=L-1$ [1, 9], whose recurrence is Theorem 3.7 of [1] (equivalent to the differential equation above; we found no general-ℓ generating function in the literature, but the Riccati/Bernoulli route is that of [2]), and the block decomposition into almost-full tables is that of [2], who use it for a different process (failed keys are thrown away and redrawn) and a different quantity; what is new here is the ring with empty cells, the identification with the refusal-free fill, and the closed form. Table 3 shows what it buys. For a fill to load 3/4 the exact limit is close to the one given by the sum $\sum_t R_t$ and well below the one given by $N\,R_m(n_{\max},L)$: at $m=4096$ and a whole-fill failure probability 10^{-6} it is 306 against 402 for the bound of Proposition 9 in its first form, a saving of 24% in the probes allowed per call. The bound is not exact because refusals cluster: once a window of occupied cells exists, several pushes can hit it; the ratio of the true probability to the sum is 0.6–0.9 in the table.

4.1 How the law was checked

Four independent checks support Theorem 7. (i) *Exhaustive enumeration:* for every ring size $m\le9$ and every n,L with $m^n\le2\cdot10^6$ homes sequences (258 triples (m,n,L)), formula (1) evaluated in exact rational arithmetic equals the probability obtained by simulating linear probing on *every* sequence of homes: 0 mismatches. (ii) *The stored simulation:* the repository's

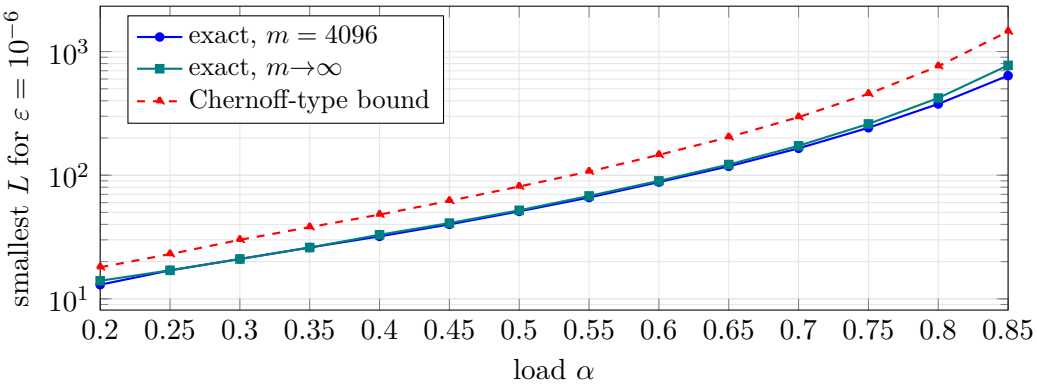


Figure 2: Probe limit required for a per-insertion refusal probability of 10^{-6}.

Table 5: Exact law against the real table. m: index slots, n: keys already stored, L: probe limit. "Clean" trials are those in which no earlier insertion exceeded L (used for the lockstep check).

m	n	L	observed	exact	z	clean trials	mismatches
256	128	8	0.03311	0.03315	-0.13	254 871	0
256	160	16	0.02643	0.02667	-0.94	307 410	0
256	190	24	0.05740	0.05730	+0.26	247 303	0
256	190	40	0.01372	0.01375	-0.19	369 939	0
1024	760	48	0.01251	0.01286	-1.96	307 286	0
1024	760	96	0.00060	0.00058	+0.52	397 066	0
1024	512	24	0.00040	0.00046	-1.63	395 607	0

240-million-sample simulation of fresh insertions into tables of 65 536 slots agrees with the exact law wherever the simulation observed events (e.g. 7.71×10^{-3} against 7.70×10^{-3} at $L = 64$, load 3/4). (iii) *The real implementation:* Table 5 compares the law with the actual STRATA table (keyed hash with a random key per trial, static index, probe limit): the observed frequency of a fresh name needing more than L probes in 400 000 independent trials per row. The largest $|z|$ is 1.96. In the same trials a second table with the probe limit set and an identical key was kept in lockstep: while no earlier probe exceeded L its state digest equalled that of the unlimited table, and it returned the refusal status exactly when the probe count exceeded L (0 mismatches). (iv) *An independent derivation:* counting by Abel's binomial theorem in the manner of Knuth [24] gives the probability that a run of exactly g occupied cells starts at a fixed cell as $\binom{n}{g}(g+1)^{g-1}(k-1)(m-g-1)^{n-g-1}/m^n$, hence $\sum_{g \geq L}(g-L+1)P_g$ for the refusal probability; this shares no step with the proof in Appendix A and agrees with formula (1) in 481 cases (exactly for $m \leq 14$, to relative difference 2e-10 for m up to 2^{20}).

5 Keys and Adversaries

Theorem 7 assumes independent uniform homes. Against names chosen by an adversary this is a statement about the hash, and a statement about *what the adversary may observe*. We make this precise.

Adversary model. The table's hash is a random function F_K drawn from a secret key K (a pseudorandom function is modelled as ideal). An adversary chooses the names that are declared before the honest ones. A *strong oracle* returns F_K(name) for a queried name; a *weak oracle* returns one bit, whether the home of the queried name equals the home of a fixed anchor name (this is what a perfect timing channel on a probe-limited table reveals: the declaration is slower

Table 6: Honest names refused by the probe limit (percent of 700): the adversary's learned colliders declared first. Mean over 5 seeds; maximum over seeds for the persistent key.

colliders	persistent key (mean)	(max)	fresh key	random names
400	1.00%	1.29%	0.00%	0.00%
512	5.51%	6.14%	0.00%	0.00%
600	4.71%	6.00%	0.00%	0.00%
700	5.14%	5.71%	0.00%	0.00%
800	5.14%	6.00%	0.00%	0.00%

exactly when the probe sequence is longer).

Proposition 11 (Fresh key)**.** *Let every table be created with a key drawn independently of everything the adversary has seen, and let the adversary fix its names before the key is drawn. Then the homes of all distinct names, honest or not, are independent and uniform, and Theorem 7 and Proposition 9 hold verbatim: whatever names the adversary declares and for every scope schedule fixed before the key is drawn (a schedule that adapts to what the adversary observes in the same table, for instance through a timing channel, is the weak-oracle attack of Proposition 13 within one key's lifetime and is not covered), the refusal probability of an honest name is at most* $R_m(n_{\max}, L)$*.*

Proof. In the ideal model F_K for an independent K is a uniformly random function; an oblivious choice of names cannot correlate with it. The remaining statements are Theorem 7 and Corollary 8(a) applied to the live count, which the adversary can only increase up to $n_{\max}$. □

Proposition 12 (Persistent key, strong oracle)**.** *Suppose the same key is used for many tables and the adversary has a strong oracle. After* $q \geq m(\ln m + t)$ *queries of random names, with probability at least* $1 - e^{-t}$ *the adversary knows a name for every cell. Declaring names for* $r \leq n_{\max}$ *consecutive cells first, in an otherwise empty table, produces a run of* r *occupied cells without any refusal, and then the next honest name is refused with probability at least* $(r - L + 1)/m$ *when* $r \geq L$*. For* r *well above* L *this is far above the exact law (Theorem 7).*

Proof. A fixed cell is missed by q independent uniform queries with probability $(1 - 1/m)^q \leq e^{-q/m}$; a union over m cells gives $me^{-q/m} \leq e^{-t}$. Names for r consecutive cells each go to their own home, so the run occupies exactly those cells. An honest name whose home is one of the first $r - L + 1$ cells of the run finds L occupied cells. □

Proposition 13 (Weak oracle)**.** *With a weak oracle each query reveals a collider of the anchor with probability* $1/m$ *independently, so finding* c *colliders takes* cm *queries in expectation. A probe-limited table can store at most* $L - 1$ *of them at the anchor's home, because the* j*-th needs* $j + 1$ *probes.*

Illustration. Table 6 runs the weak-oracle attack against the real STRATA table with a secret persistent key ($m = 2048$, $L = 512$, 700 honest names after the adversary's names, 5 seeds). Learning one collider cost on average 2071 queries, as the geometric argument predicts ($\approx m$). With the same key, between 1% and 5.5% of the honest names are refused even though memory is available; names the attacker learned are useless against a table with a fresh key, and the same number of random names causes no refusal. The oracle here is the library's `longest_probe` statistic, an idealised stand-in for a timing channel; this is an illustration of Proposition 13, not a timing attack.

Key-lifetime rule. Propositions 11–12 say the guarantee of the probe limit against adversarial *names* requires that no information about a key's behaviour leaks into names that are declared in a table that uses the same key. In practice: derive the key of the i-th table from a device secret and a never-repeating counter, $K_i = \mathrm{PRF}(K_0, i)$, and never reuse a counter value (a device that resets its counter reuses keys). What does not depend on the key is the *work bound*: a call examines at most L cells whatever the names, so the worst case of hash flooding [4, 10] is bounded even when the key leaks; only the absence of refusals depends on it. Adaptivity against hash tables through timing is an old observation [27, 30]; the point here is the exact accounting of what it costs against a probe-limited table.

6 An Exact Space Ledger

The layouts compared are single-buffer scoped tables whose size at an instant is a sum of word counts. For a name of ℓ bytes let $w_4(\ell) = \lceil \ell/4 \rceil$ (raw words) and $w_5(\ell) = \lceil \ell/5 \rceil$ (five six-bit characters per word, for names of letters, digits and underscore). At an instant let d be the number of live declarations, n the number of distinct visible names, $h = d - n$ the hidden declarations, s the open scopes, m the index slots (a power of two, at least 16, doubled when $4(n+1) > 3m$, never reduced within a translation unit) and b the buckets of a chained table (doubled when $d + 1 > b$). The five layouts are:

layout	words in use	per-element costs
C_0 chained, stored hash (baseline)	$b + 1 + \sum_{\mathrm{decl}} (4 + w_4) + s$	link, hash, value, meta, name
C_1 chained, no stored hash	$b + 1 + \sum_{\mathrm{decl}} (3 + w_4) + s$	link, value, meta, name
C_2 open-addressed, entry per declaration	$m + 1 + \sum_{\mathrm{decl}} (3 + w_4) + s$	prev, value, meta, name
S_{raw} interned, raw names	$m + 1 + \sum_{x} (2 + w_4) + n + 3h + 2s$	record per name, undo
S interned, packed names (STRATA)	$m + 1 + \sum_{x} (2 + w) + n + 3h + 2s$	$w = w_5$ if packable else w_4

Only C_0, S_{raw} and S exist as code in the repository; C_1 and C_2 are layout rules, evaluated by accounting alone.

Proposition 14 (Ledger)**.** *At every instant, with the same m and b as the layouts would have at that instant,*

$$U_{C_0} - U_{C_1} = d, \qquad U_{C_1} - U_{C_2} = b - m,$$
$$U_{C_2} - U_{S_{\mathrm{raw}}} = \sum_{\textit{hidden decl.}} w_4(\ell) \;-\; s, \qquad U_{S_{\mathrm{raw}}} - U_S = \sum_{x \textit{ visible, packable}} \big(w_4(\ell_x) - w_5(\ell_x)\big).$$

Proof. Direct subtraction of the formulas above: C_0 and C_1 differ by one word per declaration; S_{raw} charges $3 + w_4$ per first declaration (record $2 + w_4$ plus one undo word) and 3 per hidden declaration, where C_2 charges $3 + w_4$ for every declaration, and one scope mark costs 2 words in S against 1 in C_2; packing changes w_4 to w_5 for packable names only. □

Hence the saving of the compact layout over a conventional one has an exact anatomy: a stored hash costs exactly one word per declaration; interning with an undo log pays only through the *names of hidden declarations* and costs one extra word per open scope; and packing saves $\lceil \ell/4 \rceil - \lceil \ell/5 \rceil$ words per name, about $\ell/20$.

Table 7 evaluates the ledger on the traces. The compact layout needs 16.4–17.9% less than the conventional chained table, of which the stored hash word accounts for 9.2–12.2% and packing for 5.3–9.0% (measured against C_1). Interning with an undo log changes the footprint by between 0.00% and −0.38% relative to the open-addressed layout C_2 (negative: larger, because a scope mark costs two words): the programs shadow little, and Proposition 14 says why — the interned layout saves $\sum_{\mathrm{hidden}} w_4 - s$ words. This corrects an impression that the earlier empirical comparison could leave: against a conventional table that stores a hash, the

Table 7: Smallest fixed buffer (KiB) for the five layouts on the eight compiler traces of the repository, and the savings (percent): of dropping the stored hash ($C_0 \to C_1$), of packing ($C_1 \to S$), and in total ($C_0 \to S$). The accounting reproduces all 24 minimum buffers measured for C_0, S_{raw} and S exactly (to the 64-byte resolution of the measurement).

trace	C_0	C_1	C_2	S_{raw}	S	hash	pack	total
zlib	18.9	16.6	16.6	16.7	15.8	11.9	5.3	16.6
cJSON	35.9	31.8	31.8	31.8	30.0	11.5	5.5	16.4
littlefs	24.2	21.4	21.4	21.5	20.0	11.6	6.7	17.5
FreeRTOS	53.6	47.8	47.8	47.8	44.0	11.0	7.9	17.9
Lua	43.4	38.1	38.1	38.1	35.9	12.2	5.7	17.3
Mbed TLS	96.9	87.9	87.9	87.9	80.0	9.2	9.0	17.4
TinyCC	189.8	168.2	168.2	168.3	156.5	11.3	7.0	17.5
SQLite	169.4	152.8	152.8	152.9	140.4	9.8	8.1	17.2

compact layout looks one sixth smaller, but that margin is the stored hash plus packing, not the interning or the undo entries. The ledger does not say what happens on programs with heavy shadowing; the identity does: the saving then grows linearly in the number and length of hidden names.

7 Related Work

Scoped symbol tables. The undo-stack scheme is described by Appel [3] and goes back to hashed tables with explicit scope control [16]; LLVM's `ScopedHashTable` [28] implements it over an allocator. We read the symbol table of lcc [15, 18], the closest arena-based design: names are interned strings hashed by address, a table is created lazily per scope level and linked to the enclosing one by a `previous` pointer, lookup walks that chain outward, and leaving a scope pops the table pointer; storage comes from arenas [17] released at function end. It has neither a single index nor an undo log, a lookup may have to search every enclosing scope table, and it has no budget or refusal semantics.

Linear probing and deletions. The analysis of linear probing [14, 20, 21, 24, 33, 37] gives the cost distributions for insertion-only tables; our Theorem 7 is the tail of the unsuccessful-search cost, whose exact generating function and Poisson limit are in [22]; we contribute an elementary derivation of the explicit finite-size sum and its use as a refusal probability, not the distribution itself. For deletions, Knuth's backward shifting and tombstones [23, 24], the behaviour of tombstones [5], and tombstone minimisation with stable positions [34] address arbitrary orders; the state of the art for open addressing without reordering is [12] (insertions) and [6] (the lower bound for dynamic workloads), with a follow-up on greedy open addressing in 2026 [13]. Dynamic linear probing has been analysed under random deletions in the uniform-hashing model [26]. None of these treats the stack-ordered case; a few web searches ("LIFO deletion open addressing", "undo/rollback hash table") found no statement of Theorems 2–4, which is weaker evidence than a systematic search and may reflect folklore in backtracking implementations. Undoing a hash-table update on backtracking is routine (for example, the open-addressed `library(hashtable)` of SWI-Prolog restores cells from its trail in last-in-first-out order), but we found no statement of the property as a theorem for a class of schemes. History-independent hashing [7, 29] makes the layout a function of the stored *set*; the canonical state of Theorem 2 is a function of the stored *sequence*, which is weaker.

Bounded probe sequences. Alharbi, Banderier and Bouillaguet [2] study a bounded-probe linear-probing *process* in which a key whose probe sequence exceeds p is discarded and another

is drawn, and derive the expected number of attempts needed to fill the table; they list the probability that an insertion succeeds at a given fill ratio as a question left open. That is a different quantity from ours: Theorem 7 gives the refusal probability of an insertion into the table that the unlimited process would have built, which equals the refusal probability of the bounded process up to its first refusal (Proposition 9); after a refusal the two processes differ and we make no claim. Cuckoo [32] and hopscotch [19] hashing bound probes by moving keys, which would break the undo property. A patent describes a cache with probe-limited open addressing that evicts or refuses an insertion at the limit [31]; to our knowledge no analysis of the refusal probability of such tables has been published.

Hash flooding and adaptivity. Algorithmic complexity attacks [10] and seed-independent multicollisions of non-cryptographic hashes [4] motivate keyed hashing; timing as an oracle for collisions [27] and adversarial Bloom filters [30] are the adaptive-adversary precedents. Our contribution here is the accounting for a probe-limited table.

8 Limitations

- **Ideal hashing.** Theorem 7 and Corollary 3 assume independent uniform homes. Real identifiers are not random strings; the repository's compiler traces are consistent with uniform spreading but do not prove it. A keyed hash is treated as an ideal function; HalfSipHash-1-3 with a 64-bit key is documented as adequate for hash-table flooding only.
- **Orphan law: limit by a local argument.** Proposition 5(b) is proved by a window argument and the Poisson limit of home counts; the rate of convergence ($O(1/m)$ in the checks) is not proved. The law concerns clearing one key from a table built by insertions, not a sequence of out-of-order deletions.
- **Trajectories.** Theorem 10 is exact for a monotone fill from empty. For a general push/pop schedule only the union bound of Proposition 9 is available; the number (not just the occurrence) of refusals in a run, and schedules that oscillate near a fixed table, are not derived.
- **The law is a known quantity.** Theorem 7 and Corollary 8(b) restate classical results (Remark 2); the new parts are the Chernoff gap, the design rule, the validation against an implementation, and the link to stack discipline (Corollary 3).
- **Two probing rules in the refusal law.** Theorems 2 and 4 cover the general class and greedy schemes respectively; the refusal probability is exact for linear probing (Theorem 7) and for uniform probing (a one-line product), not for quadratic probing or double hashing.
- **The probe-complexity theorem leans on published bounds.** Theorem 6(ii) combines the exact transfer of Theorem 2 with the upper and lower bounds of [12], whose proofs we read but did not re-derive, under their conditions on δ; part (iii) quotes a reduction from tiny pointers that we did not re-prove. It concerns expected probe indices in the random-function model against oblivious workloads.
- **Idealised oracle.** The attacks of Section 5 use a model oracle. No timing measurement was made, and whether a given device leaks enough timing to realise it is not studied.
- **Ledger.** The five layouts are accounting rules; C_1 and C_2 were not implemented, so their speed is unknown. The conclusions about interning hold for the eight C traces and the stated identity; heavily shadowing code is not covered by the data.

- **Verification.** The proofs are on paper. Exhaustive enumeration ($m \leq 9$), randomized checks and the statistical test against the implementation support them; none replaces a machine-checked proof.
- **Literature.** The search was limited (a modest number of web searches, Crossref verification of every entry, primary sources read where stated in the bibliography notes). A closer precedent may exist.
- **Scope of the application.** Single-threaded tables, names of at most 255 bytes, one name space per table; no hardware was used.

9 Conclusion

The last-in-first-out order of scope exit is not just convenient for a scoped symbol table; it is the only deletion order for which clearing is safe whatever the hashes, and for a given table the safe orders are exactly the linear extensions of an explicit dependency relation, it makes the table's state a function of the live sequence alone, and it turns the analysis of a table with deletions into the classical insertion-only analysis. Using this we gave an exact refusal law for probe-limited tables, tighter probe limits than a Chernoff-type argument, a precise statement of what a key must be and how long it may live, and a ledger showing where a compact layout's memory saving actually comes from. Natural next steps are machine-checked proofs of Theorems 2–7, the exact law for other probing rules, a trajectory-level statement for schedules other than a monotone fill, and measurements of the timing channel on a device.

Declarations

Use of AI assistance. The software, tests, benchmarks and the earlier empirical manuscript that this article extends were produced by an AI coding assistant working in the environment of the first author, under the authors' direction, as recorded in the repository (`docs/CONTRIBUTIONS.md`). This article was developed with the same kind of tool: Anthropic's Claude (model Sonnet 5.5, through the Claude Code command-line tool) was used, at one author's direction, to review the earlier work adversarially, to propose and derive the theorems and proofs of Sections 3–6, to write the verification programs in `theory/verify/`, and to draft the text. Every number in the article is produced by those programs, which can be re-run (Appendix B). The tool is not an author; the listed authors are responsible for the content.

Data and code availability. All sources, programs, traces and raw results are in the public repository [35], directory `theory/` for this article.

References

[1] Kimberly Aguilar-Fraga, Jennifer Elder, Rebecca E. Garcia, Kimberly P. Hadaway, Pamela E. Harris, Imhotep B. Harry, et al. Interval and ℓ-interval rational parking functions. *Discrete Mathematics & Theoretical Computer Science*, 2024. doi: 10.46298/dmtcs.12598.

[2] Ahmed Alharbi, Cyril Banderier, and Charles Bouillaguet. Bounded linear probing hashing. In *37th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms (AofA 2026)*, volume 381 of *LIPIcs*, pages 28:1–28:17, 2026. doi: 10.4230/LIPIcs.AofA.2026.28. Abstract, introduction, Section 3 and conclusion read.

[3] Andrew W. Appel. *Modern Compiler Implementation in C*. Cambridge University Press, Cambridge, UK, 1998. doi: 10.1017/CBO9781139174930.

[4] Jean-Philippe Aumasson and Daniel J. Bernstein. SipHash: A fast short-input PRF. In *Progress in Cryptology – INDOCRYPT 2012*, volume 7668 of *Lecture Notes in Computer Science*, pages 489–508. Springer, 2012. doi: 10.1007/978-3-642-34931-7_28.

[5] Michael A. Bender, Bradley C. Kuszmaul, and William Kuszmaul. Linear probing revisited: Tombstones mark the demise of primary clustering. In *Proc. 62nd IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 1171–1182, 2021. doi: 10.1109/FOCS52979.2021.00115.

[6] Michael A. Bender, Alex Conway, Martín Farach-Colton, William Kuszmaul, and Guido Tagliavini. Tiny pointers. *ACM Transactions on Algorithms*, 21(4):1–43, 2025. doi: 10.1145/3700594. Theorem numbers refer to the arXiv version 2111.12800, which was read.

[7] Guy E. Blelloch and Daniel Golovin. Strongly history-independent hashing with applications. In *Proc. 48th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 272–282, 2007. doi: 10.1109/FOCS.2007.36.

[8] Pedro Celis, Per-Åke Larson, and J. Ian Munro. Robin Hood hashing. In *Proc. 26th Annual Symposium on Foundations of Computer Science (SFCS)*, pages 281–288, 1985. doi: 10.1109/SFCS.1985.48.

[9] Kimberly Colaric, Ryan DeMuse, Jeremy L. Martin, and Mei Yin. Interval parking functions. *Advances in Applied Mathematics*, 123:102129, 2021. doi: 10.1016/j.aam.2020.102129.

[10] Scott A. Crosby and Dan S. Wallach. Denial of service via algorithmic complexity attacks. In *Proc. 12th USENIX Security Symposium*, 2003.

[11] M. Dwass. The total progeny in a branching process and a related random walk. *Journal of Applied Probability*, 6(3):682–686, 1969. doi: 10.2307/3212112.

[12] Martin Farach-Colton, Andrew Krapivin, and William Kuszmaul. Optimal bounds for open addressing without reordering. In *Proc. 65th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 594–605, 2024. doi: 10.1109/FOCS61266.2024.00045. Statements used here were read in the arXiv version 2501.02305.

[13] Martin Farach-Colton, Andrew Krapivin, and William Kuszmaul. Greedy open addressing revisited: Beyond Yao's lower bound. In *Proc. 58th ACM Symposium on Theory of Computing (STOC)*, pages 1116–1127, 2026. doi: 10.1145/3798129.3800823. Record checked in Crossref; not read.

[14] P. Flajolet, P. Poblete, and A. Viola. On the analysis of linear probing hashing. *Algorithmica*, 22(4):490–515, 1998. doi: 10.1007/PL00009236.

[15] Christopher W. Fraser and David R. Hanson. *A Retargetable C Compiler: Design and Implementation*. Addison-Wesley, Redwood City, CA, 1995. ISBN 0-8053-1670-1.

[16] S. L. Graham, W. N. Joy, and O. Roubine. Hashed symbol tables for languages with explicit scope control. In *Proc. SIGPLAN Symposium on Compiler Construction*, pages 50–57, 1979. doi: 10.1145/800229.806953.

[17] David R. Hanson. Fast allocation and deallocation of memory based on object lifetimes. *Software: Practice and Experience*, 20(1):5–12, 1990. doi: 10.1002/spe.4380200104.

[18] David R. Hanson and Christopher W. Fraser. lcc, file `src/sym.c`. `https://github.com/drh/lcc`, repository head of 2024-10-06 (2b5cf358d9). Read 2026-10-17.

[19] Maurice Herlihy, Nir Shavit, and Moran Tzafrir. Hopscotch hashing. In *Distributed Computing (DISC 2008)*, Lecture Notes in Computer Science, pages 350–364. Springer, 2008. doi: 10.1007/978-3-540-87779-0_24.

[20] Svante Janson. Asymptotic distribution for the cost of linear probing hashing. *Random Structures & Algorithms*, 19(3–4):438–471, 2001. doi: 10.1002/rsa.10009.

[21] Svante Janson. Individual displacements for linear probing hashing with different insertion policies. *ACM Transactions on Algorithms*, 1(2):177–213, 2005. doi: 10.1145/1103963.1103964.

[22] Svante Janson and Alfredo Viola. A unified approach to linear probing hashing with buckets. *Algorithmica*, 75(4):724–781, 2016. doi: 10.1007/s00453-015-0111-x. Section 11 read in the arXiv version 1410.5967.

[23] Rosa M. Jiménez and Conrado Martínez. On deletions in open addressing hashing. In *Proc. 15th Workshop on Analytic Algorithmics and Combinatorics (ANALCO)*, pages 23–31, 2018. doi: 10.1137/1.9781611975062.3.

[24] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2nd edition, 1998.

[25] Alan G. Konheim and Benjamin Weiss. An occupancy discipline and applications. *SIAM Journal on Applied Mathematics*, 14(6):1266–1274, 1966. doi: 10.1137/0114101.

[26] Per-Åke Larson. Analysis of uniform hashing. *Journal of the ACM*, 30(4):805–819, 1983. doi: 10.1145/2157.322407.

[27] Richard J. Lipton and Jeffrey F. Naughton. Clocked adversaries for hashing. *Algorithmica*, 9(3):239–252, 1993. doi: 10.1007/BF01190898.

[28] LLVM Project. ScopedHashTable.h, LLVM ADT library. `https://llvm.org/doxygen/ScopedHashTable_8h_source.html`. Accessed 2026-10-13.

[29] Moni Naor and Vanessa Teague. Anti-persistence: History independent data structures. In *Proc. 33rd ACM Symposium on Theory of Computing (STOC)*, pages 492–501, 2001. doi: 10.1145/380752.380844.

[30] Moni Naor and Eylon Yogev. Bloom filters in adversarial environments. In *Advances in Cryptology – CRYPTO 2015*, Lecture Notes in Computer Science, pages 565–584. Springer, 2015. doi: 10.1007/978-3-662-48000-7_28.

[31] Richard M. Nemes, Mikhail Lotvin, and David Garrod. Methods and apparatus for information storage and retrieval using a caching technique with probe-limited open-address hashing. US Patent 9,104,678 B1, 2015. First 100 000 characters of the web page read; claims not read.

[32] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004. doi: 10.1016/j.jalgor.2003.12.002.

[33] B. Pittel. Linear probing: The probable largest search time grows logarithmically with the number of records. *Journal of Algorithms*, 8(2):236–249, 1987. doi: 10.1016/0196-6774(87)90040-X.

[34] Peter Sanders. Hashing with linear probing and referential integrity. arXiv:1808.04602, 2018. Full text read; it contains no statement of stack-ordered deletion.

[35] Shubhi Singh and Adhyan Jain. STRATA: a budgeted, stack-disciplined scoped symbol table (repository `cdesproject`). `https://github.com/shubhisingh1510/cdesproject`. Directory `theory/` holds this article's sources and verification programs.

[36] J. C. Tanner. A derivation of the Borel distribution. *Biometrika*, 48(1–2):222–224, 1961. doi: 10.1093/biomet/48.1-2.222.

[37] A. Viola. Exact distribution of individual displacements in linear probing hashing. *ACM Transactions on Algorithms*, 1(2):214–242, 2005. doi: 10.1145/1103963.1103965.

[38] Andrew Chi-Chih Yao. Uniform hashing is optimal. *Journal of the ACM*, 32(3):687–693, 1985. doi: 10.1145/3828.3836.

A Proof of Theorem 7

Throughout, $k = m - n \geq 1$; the case $k = 1$ is direct (exactly one cell is empty and it is uniformly located, so the L cells from a uniform start are occupied iff the start lies in the $(m-L)^+$ cells that precede the empty cell by at least L). Let X_j be the number of keys with home j; (X_j) is multinomial with $\mathbb{P}(X = x) = \frac{n!}{m^n} \prod_j \frac{1}{x_j!}$ on $\{\sum x_j = n\}$.

Step 1: blocks. Since $n < m$ there are exactly k empty cells and the occupied set depends only on x, not on the insertion order [25]. Cut the ring at the empty cells: a *block* is an empty cell together with the occupied cells that follow it up to, but excluding, the next empty cell; a block of g cells has one empty cell and $g-1$ occupied ones. No key crosses an empty cell (it would stop there), so the keys homed in a block occupy exactly the cells of that block, $g-1$ of them. Writing $x_1 = 0, x_2, \ldots, x_g$ for the home counts of the cells of the block, the cells $2, \ldots, g$ are occupied and the next cell is empty iff

$$y_u := x_{u+1} \quad (1 \leq u \leq g-1) \quad \text{satisfy} \quad S_u := \sum_{t \leq u} y_t \;\geq u \;\; (u < g-1), \qquad S_{g-1} = g-1. \tag{2}$$

Conversely, any ring configuration that is a cyclic concatenation of blocks with patterns (2) has exactly these empty cells (the recursion $Y_c = (Y_{c-1} - 1)^+ + X_c$ has a unique solution on a ring when $n < m$, since some Y_c is 0). So home-count vectors with $\sum x_j = n$ correspond bijectively to a placement of k blocks around the ring, each with a pattern (2).

Step 2: weight of a block. Append $y_g := 0$ and put $z_t = y_t - 1 \geq -1$. Condition (2) says that the walk $z_1 + \cdots + z_u$ stays ≥ 0 for $u < g$ and first reaches -1 at $u = g$. By the cycle lemma, among the g cyclic rotations of any sequence in $\mathbb{N}^g$ with $\sum y = g-1$ exactly one has this property (no such sequence has a nontrivial period, since the sum -1 is not divisible by g/d for a period $d < g$). The weight $\prod_t 1/y_t!$ is rotation invariant, and $\sum_{y \in \mathbb{N}^g,\, \sum y = g-1} \prod_t \frac{1}{y_t!} = \frac{g^{g-1}}{(g-1)!}$ by the multinomial theorem. Hence the total weight of the patterns of a block of g cells is $\frac{1}{g} \frac{g^{g-1}}{(g-1)!} = B(g)$. (With the factors $e^{-\theta}$ of an independent Poisson(θ) model this is exactly Borel($g; \theta$), the total progeny of a Poisson(θ) branching process [11].)

Step 3: the block containing a cell. The probability of a home-count vector is proportional to $\prod_j 1/x_j!$, so the probability weight of a ring configuration is proportional to the product of the block weights. Count configurations in which the block containing cell 0 has g cells: cell 0 occupies one of the g positions of that block, and the other $k-1$ blocks follow in order with $m - g$ cells in total, contributing $\sum_{g_2 + \cdots + g_k = m-g} \prod B(g_i) = Q_{k-1}(m-g)$. By

exchangeability, $\sum_g g\,B(g)Q_{k-1}(m-g) = \frac{m}{k}Q_k(m)$ (multiply $Q_k(m) = \sum\prod B(g_i)$ by $m = \sum g_i$ and use symmetry). Therefore

$$\mathbb{P}(\text{block of cell 0 has } g \text{ cells}) = \frac{k}{m}\,g\,\pi_g, \qquad \pi_g = \frac{B(g)Q_{k-1}(m-g)}{Q_k(m)},$$

and, given g, the position of cell 0 in the block is uniform on $\{1,\dots,g\}$.

Step 4: the distance to the next empty cell. Let D be the number of steps from cell 0 to the first empty cell at or after it ($D = 0$ if cell 0 is empty). For position $o \in \{2,\dots,g\}$ in a block of g cells, the next empty cell is the first cell of the next block, at distance $g+1-o$, so $D \geq L$ iff $o \leq g+1-L$, which holds for $\max(g-L,0)$ positions. Position 1 has $D = 0 < L$. Thus

$$\mathbb{P}(D \geq L) = \sum_g \frac{k}{m} g\,\pi_g \cdot \frac{(g-L)^+}{g} = \frac{k}{m}\sum_{g>L}(g-L)\pi_g.$$

A new key with an independent uniform home sees the first L cells all occupied iff $D \geq L$ from its home; by rotation invariance this is $\mathbb{P}(D \geq L)$ at cell 0. The range $g \leq n+1$ holds because a block has at most n occupied cells. □

B Reproducing every number

Requirements: `g++` with C++14, `python3` (standard library only), `pdflatex` with `latexmk` and `pgfplots`. From the repository root,

```
bash theory/verify/run_all.sh
```

runs, in order: `exact_law.py` (law against brute-force enumeration in exact rationals), `fill_law.py` (Theorem 10 against brute force over every home sequence and against its closed form, and Table 3), `orphan_law.py` (the characterisation of safe orders and the orphan law of Proposition 5 against enumeration, and the limit law), `lifo_check.py` (Theorems 2–4 and Proposition 5 on five insertion rules, exhaustive over all home assignments and deletion orders for $m \leq 8$ as stated in Remark 1), `design_table.py` (law against the stored simulation), `law_vs_table.cpp` with `run_law_check.py` (law against the real table), `design_rule.py`, `proof_checks.py` (the ledger identities at 848 449 instants of the eight traces, the limit and tail-asymptotic corollaries, and the orphan mean: measured 1.50 against 1.50 at load 3/4), `space_model.py` (ledger and the check against the measured minima), `adaptive_key_demo.cpp`, and `make_figdata.py`, then builds this PDF. All tables, figure data and the numeric macros of the text are written into `theory/paper/tables` and `theory/paper/data` by these programs. The traces and the measured minima are those of the repository (`workloads/traces`, `results/raw/20261013-143828-paper`). Approximate run time: eight minutes.

The stack-discipline checks of `lifo_check.py` produced 488 919 state comparisons for Theorem 2 with 0 failures, the exhaustive check of Theorem 4(b) (18 settings: every assignment of homes and every deletion order, linear probing with $m \in \{4,5,6\}$ and all $n \leq m$, quadratic probing with $m \in \{4,8\}$ and $n \leq 4$; part (a) additionally on 397 562 pairs for $m \leq 5$) 0 failures, and the orphan identity of Proposition 5 (all assignments for four small (m,n)) 0 failures.